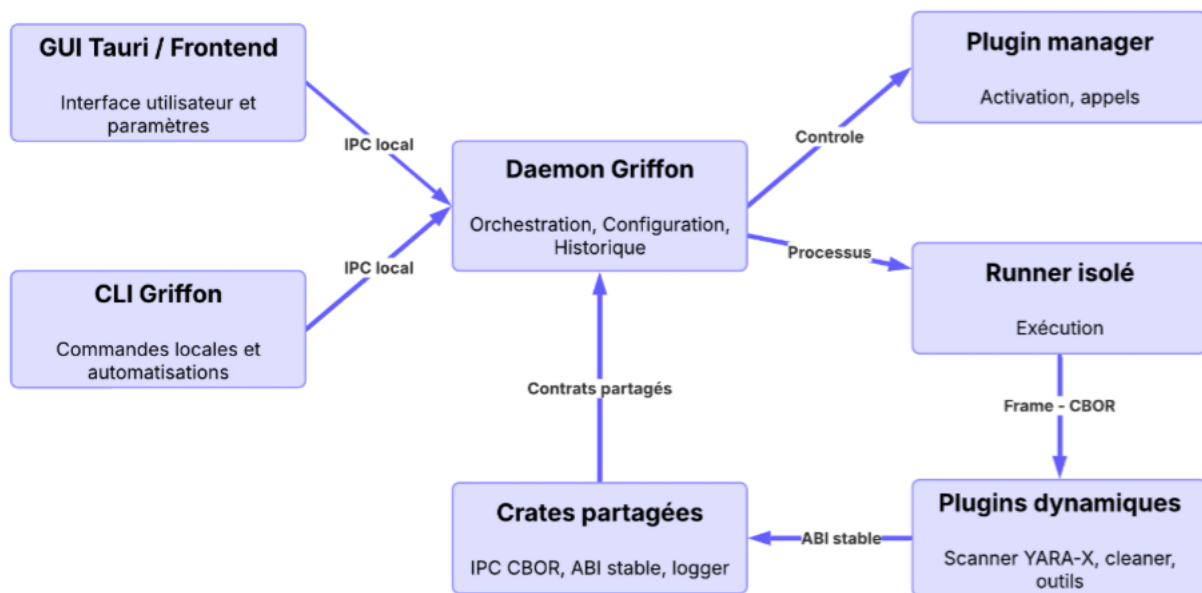


# DOSSIER KPI 1

## Évaluer et intégrer de nouvelles technologies

### Projet Griffon — Plateforme de sécurité modulaire pour Linux



Architecture synthétique mise en œuvre dans Griffon

#### Version condensée — juillet 2026

*Veille • Comparaison • POC • Intégration • Mesure • Partage*

# 1. Synthèse exécutive

**Objectif : démontrer que Griffon a identifié des besoins techniques réels, comparé plusieurs solutions, expérimenté, intégré les choix retenus et mesuré leur impact.**

**Griffon est passé d'un prototype monolithique à une plateforme composée d'un daemon, d'une GUI Tauri, d'une CLI et de plugins dynamiques.** La veille a directement guidé cinq choix structurants : ABI stable, IPC Frame + CBOR, YARA-X, Tauri et Grafana/SQLite.

| Besoin                                | Choix retenu            | Preuve concrète                                  |
|---------------------------------------|-------------------------|--------------------------------------------------|
| Charger des plugins Rust indépendants | abi_stable              | Plugin Manager, interface partagée, plugins .so  |
| Fiabiliser les échanges locaux        | Frame 12 octets + CBOR  | Hello/Call/Result/Error, request_id, limite 1 Mo |
| Intégrer un moteur de détection       | YARA-X                  | Scanner Rust, règles, archives, quarantaine      |
| Fournir une application desktop       | Tauri                   | GUI connectée au daemon et aux plugins           |
| Mesurer les performances              | JSON → SQLite → Grafana | Comparaison Cleaner v0.0.9 / v0.1.0              |

## Résultats clés

- Deux POC structurants adoptés : Plugin Manager/ABI et protocole IPC v1.
- YARA-X intégré à la place d'une intégration directe de YARA C.
- Cleaner v0.1.0 : +8,3 % sur deux runs safe comparables fournis.
- Incident des 10 000 chemins corrigé par tri, agrégation et limitation des payloads.
- Veille régulière via RSS/Discord et sollicitation de la communauté Rust sur Reddit.

**Conclusion : le KPI est couvert par des preuves de veille, d'expérimentation, d'intégration, de mesure et de partage.**

## 2. Contexte et méthode de veille

### 2.1 Problématique

Comment construire une plateforme Linux extensible par plugins Rust, tout en gardant un contrat stable, des communications bornées, une détection performante et un processus de mesure reproductible ?

### 2.2 Périmètre surveillé

| Axe            | Sujets suivis                                               |
|----------------|-------------------------------------------------------------|
| Rust / plugins | ABI, FFI, bibliothèques dynamiques, abi_stable, WebAssembly |
| Architecture   | daemon, runner, isolation, versionnement des contrats       |
| IPC            | framing, CBOR, corrélation, limites de taille               |
| Sécurité       | YARA-X, dépendances Rust, permissions, règles               |
| Desktop        | Tauri, frontend/backend, modèle de confiance                |
| Performance    | métriques, SQLite, Grafana, régressions                     |
| Déploiement    | systemd, .deb/.rpm, droits de groupe                        |

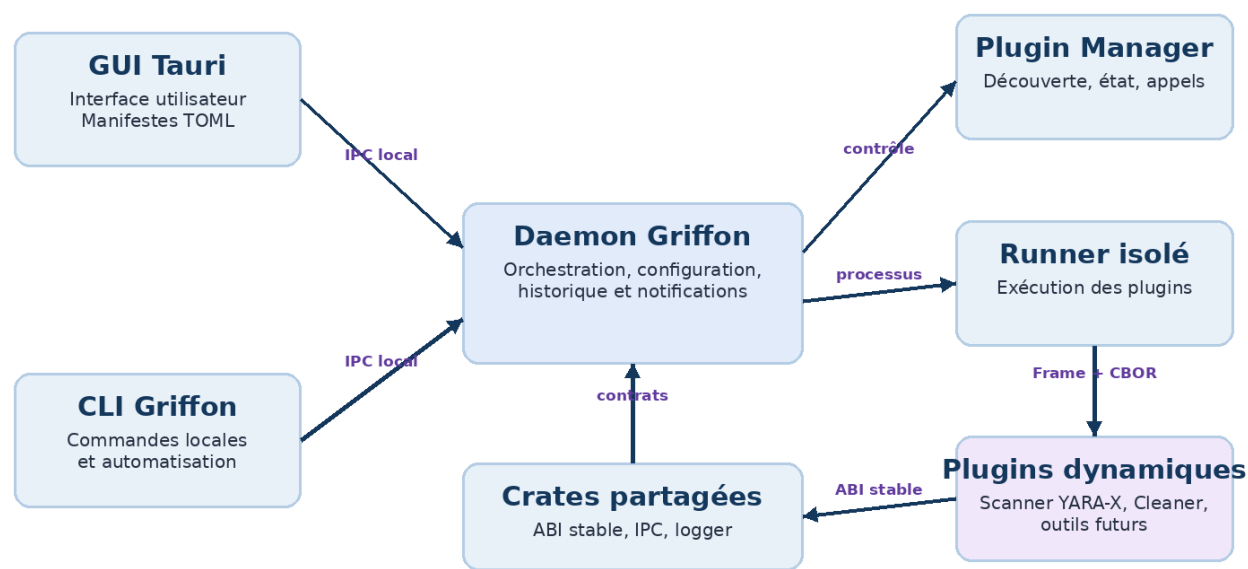
### 2.3 Processus appliqué

| Étape | Action                                                   |
|-------|----------------------------------------------------------|
| 1     | Formuler un problème lié au projet.                      |
| 2     | Consulter les sources officielles et les dépôts.         |
| 3     | Comparer les alternatives.                               |
| 4     | Réaliser un POC limité.                                  |
| 5     | Mesurer et documenter.                                   |
| 6     | Adopter, rejeter ou différer.                            |
| 7     | Intégrer via issue/PR et mettre à jour la documentation. |

### 2.4 Sources

Documentation officielle Rust, RFC 8949, abi\_stable, YARA-X, Tauri et Grafana ; dépôt GitHub Griffon ; rapports JSON et dashboard Grafana ; flux Rust Foundation, The Hacker News et This Week in Rust ; échanges Reddit.

### 3. Évolution de l’architecture



Daemon central, runner isolé et plugins dynamiques

#### Évolution vérifiable

| Date          | Trace          | Résultat                                                 |
|---------------|----------------|----------------------------------------------------------|
| Nov. 2025     | PR #90         | Première version fonctionnelle du Plugin Manager.        |
| Déc. 2025     | PR #91         | Protocole IPC v1.                                        |
| Mars 2026     | PR #118        | Nouvelle architecture du daemon, encore expérimentale.   |
| Avr. 2026     | PR #122        | Daemon de base fonctionnel.                              |
| Mai-juin 2026 | PR #149        | Packaging et exécution CLI/GUI sans sudo.                |
| Juin 2026     | PR #175 / #180 | Historique et notifications intégrés du daemon à la GUI. |

Fil conducteur : POC monolithique → architecture modulaire → packaging → instrumentation → mesure.

## 4. Décisions technologiques

### 4.1 Plugins : abi\_stable

| Option          | Avantage                                 | Limite                           | Décision    |
|-----------------|------------------------------------------|----------------------------------|-------------|
| ABI Rust native | Simple                                   | Aucune stabilité garantie        | Rejetée     |
| ABI C           | Stable et multilangage                   | Conversions et ownership manuels | Alternative |
| abi_stable      | Types ABI-safe et contrôle au chargement | Types spécifiques                | Adoptée     |
| WebAssembly     | Isolation et portabilité                 | Runtime et migration importants  | Différée    |

Le contrat est volontairement minimal : `init()` retourne les métadonnées ; `handle_message()` reçoit une commande et renvoie une réponse. Les types standard Rust ne traversent pas la frontière ABI.

### 4.2 IPC : Frame + CBOR

| Élément       | Valeur / rôle                                                                         |
|---------------|---------------------------------------------------------------------------------------|
| Header        | 12 octets : magic, version, type, request_id, payload_len                             |
| Messages      | Hello, HelloOk, Call, Result, Heartbeat, Error                                        |
| Sérialisation | CBOR via <code>serde_cbor</code>                                                      |
| Sécurité      | Validation du magic, de la version, du type et limite <code>MAX_PAYLOAD = 1 Mo</code> |

CBOR est utilisé pour le canal Plugin Manager <-> runner. JSON reste utile pour les manifests et certaines réponses lisibles. La compacité du format ne remplace pas la pagination métier.

### 4.3 Détection : YARA-X

| Critère          | YARA C                   | YARA-X                                   |
|------------------|--------------------------|------------------------------------------|
| Intégration Rust | FFI et build natif       | API Rust directe                         |
| Sécurité mémoire | Dépend du code C/wrapper | Cœur écrit en Rust                       |
| Maturité         | Très établie             | Plus récente, version à surveiller       |
| Choix Griffon    | Non retenue directement  | Adoptée avec versions verrouillées et CI |

## 4. Décisions technologiques — suite

### 4.4 Application desktop : Tauri

| Option             | Atout                       | Risque / coût                        | Décision    |
|--------------------|-----------------------------|--------------------------------------|-------------|
| Tauri              | Backend Rust + frontend web | Permissions et dépendances WebKitGTK | Adoptée     |
| Electron           | Écosystème web complet      | Runtime moins aligné avec Rust       | Non retenue |
| GTK / toolkit Rust | Interface native            | Temps de développement plus élevé    | Non retenue |

Le manifeste TOML décrit l'identité, le store, les composants et les interactions. Griffon génère ainsi l'interface d'un plugin sans créer une page React dédiée. La priorité future est de rendre ces TOML plus simples à écrire et à valider pour un nouveau contributeur.

### 4.5 Mesure : JSON, SQLite et Grafana

Chaque exécution du Cleaner produit un rapport versionné. Les données sont importées dans SQLite puis comparées dans Grafana. Grafana sert au reporting, pas à la supervision temps réel.

| Métrique               | Usage                       |
|------------------------|-----------------------------|
| files_scanned_total    | Charge parcourue            |
| files_cleaned_total    | Action réellement effectuée |
| run_duration_seconds   | Temps total                 |
| bytes_freed_per_second | Débit comparatif            |
| errors_by_type         | Robustesse et permissions   |

### 4.6 Registre de décision

| Choix            | Statut  | Réévaluation                         |
|------------------|---------|--------------------------------------|
| abi_stable       | Adopté  | Si besoin de sandbox ou multilangage |
| Frame + CBOR     | Adopté  | Lors du protocole v2                 |
| YARA-X           | Adopté  | À chaque version majeure             |
| Tauri            | Adopté  | Si contraintes WebView/packaging     |
| Grafana + SQLite | Adopté  | Si automatisation CI ou volume accru |
| WebAssembly      | Différé | Après stabilisation du SDK plugin    |

## 5. POC et intégrations concrètes

| Expérience           | Question                                           | Résultat                                                 | Décision            |
|----------------------|----------------------------------------------------|----------------------------------------------------------|---------------------|
| Plugin Manager + ABI | Peut-on charger une .so Rust indépendamment ?      | init(), découverte des fonctions et appels opérationnels | Adopté              |
| Frame + CBOR         | Peut-on transporter des messages typés et bornés ? | Header 12 octets, request_id, handshake, limite 1 Mo     | Adopté              |
| Migration YARA-X     | Peut-on éviter l'intégration C directe ?           | Scanner Rust intégré ; difficulté de versions identifiée | Adopté avec pinning |

### Preuves d'intégration

- Plugins réels et plugin template présents dans le dépôt.
- Manifestes TOML utilisés pour construire le store et la GUI.
- GUI et CLI connectées au daemon.
- Activation, notifications et historique gérés côté daemon.
- Packaging .deb/.rpm et service systemd.
- Guides dédiés : architecture plugin, backend Rust, interactions, packaging et IPC.

### Critères validés

| Critère                                | État   |
|----------------------------------------|--------|
| Plugin compilable séparément du daemon | Validé |
| Appel d'une fonction via le host       | Validé |
| Erreur de protocole contrôlée          | Validé |
| Installation documentée                | Validé |
| Mesure entre versions                  | Validé |
| Partage via documentation et GitHub    | Validé |

## 6. Benchmark du plugin Cleaner

### 6.1 Mesures exactes fournies

| Date  | Version | Mode       | Fichiers | Volume   | Durée   | Débit       |
|-------|---------|------------|----------|----------|---------|-------------|
| 03/07 | v0.0.9  | aggressive | 107 563  | 15,16 Go | 1,460 s | 10,385 Go/s |
| 03/07 | v0.0.9  | safe       | 107 563  | 13,50 Go | 1,279 s | 10,557 Go/s |
| 04/07 | v0.0.9  | aggressive | 108 793  | 15,63 Go | 1,489 s | 10,500 Go/s |
| 04/07 | v0.0.9  | safe       | 108 793  | 13,52 Go | 1,163 s | 11,628 Go/s |
| 05/07 | v0.1.0  | safe       | 110 388  | 14,29 Go | 1,135 s | 12,591 Go/s |

Comparaison exacte : 11,628 Go/s en v0.0.9 safe contre 12,591 Go/s en v0.1.0 safe, soit +8,28 %.



Dashboard Grafana — débit par version et par mode

Le dashboard agrégé suggère environ +29 %, mais ce chiffre reste une estimation visuelle tant que la requête SQL ou l'export agrégé n'est pas joint.

### 6.2 Lecture

- Le débit augmente malgré un nombre de fichiers légèrement supérieur en v0.1.0.
- Le mode aggressive nettoie des milliers de fichiers et rencontre davantage d'erreurs de permission.
- files\_cleaned\_total = 0 en safe doit être documenté comme dry-run ou absence de suppression.



## 7. Retour d'expérience : réponse de plus de 10 000 chemins

**Incident : le Cleaner renvoyait une liste massive de chemins en JSON. Le daemon pouvait tomber lors du transfert de cette réponse vers le frontend.**

| Niveau        | Cause                                          | Effet                           |
|---------------|------------------------------------------------|---------------------------------|
| Métier        | Tous les chemins étaient retournés sans limite | Croissance linéaire du résultat |
| Sérialisation | Répétition des clés et préfixes JSON           | Payload inutilement volumineux  |
| Transport     | Pas de pagination ou agrégation                | Un seul message contenait tout  |
| Frontend      | Liste difficile à afficher                     | Coût mémoire et UX dégradée     |

### Correction

- Sélection des fichiers les plus volumineux.
- Agrégation par racine ou catégorie.
- Conservation de métriques synthétiques.
- Limite maximale de payload dans l'IPC.
- Séparation entre vue de synthèse et suppression ciblée.

**Leçon : passer de JSON à CBOR ne suffit pas si le modèle de données reste non borné. La réduction de l'information avant sérialisation a été l'optimisation principale.**

### Impact

| Avant                                   | Après                      |
|-----------------------------------------|----------------------------|
| Plus de 10 000 chemins dans une réponse | Résultats triés et limités |
| Risque de crash daemon                  | Erreur ou payload contrôlé |
| Rendu frontend lourd                    | Synthèse exploitable       |
| Aucune règle métier de volume           | Agrégation + MAX_PAYLOAD   |

# 8. Documentation, veille et communauté

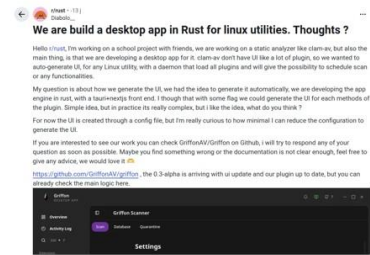
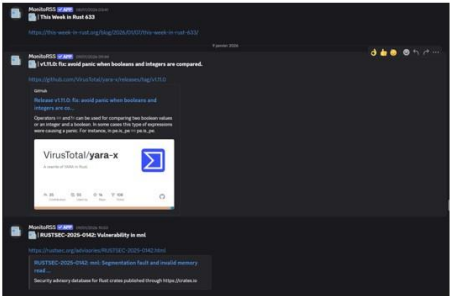
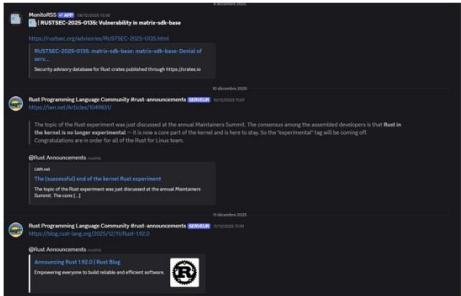
## 8.1 Capitalisation

| Document                                | Apport                                                 |
|-----------------------------------------|--------------------------------------------------------|
| README / Installation                   | Compilation, exécution, structure du dépôt et releases |
| Plugin Architecture / UI / Interactions | Modèle .so + .toml et construction de la GUI           |
| Rust Backend / Plugin Guide             | ABI, init(), handle_message() et erreurs               |
| IPC Protocol                            | Format, handshake, limites et sécurité                 |
| YARA / Grafana                          | Décisions technologiques et suivi des performances     |
| POC / PERF templates                    | Méthode commune d'expérimentation et de mesure         |

## 8.2 Veille et ouverture

### Veille et participation communautaire

|                          | STATUS | TITLE               | URL                                                                                       | ADDED ON   | SHARED WITH ME |
|--------------------------|--------|---------------------|-------------------------------------------------------------------------------------------|------------|----------------|
| <input type="checkbox"/> | 🟢      | The Rust Foundation | <a href="https://rustfoundation.org/feed/">https://rustfoundation.org/feed/</a>           | 2025-04-09 | Configure >    |
| <input type="checkbox"/> | 🟢      | The Hacker News     | <a href="https://feeds.feedburner.com/Th...">https://feeds.feedburner.com/Th...</a>       | 2025-04-09 | Configure >    |
| <input type="checkbox"/> | 🟢      | the week in rust    | <a href="https://this-week-in-rust.org/rss.xml">https://this-week-in-rust.org/rss.xml</a> | 2025-10-17 | Configure >    |



Flux RSS, relais Discord et publication r/rust

- Flux suivis : Rust Foundation, The Hacker News et This Week in Rust.
- Relais Discord : avis RUSTSEC, releases Rust et YARA-X.
- Post r/rust : question technique sur la génération de GUI et l'architecture plugin.
- Dépôt public, Apache-2.0, guides et appels à contribution.

## 9. Impact et conformité au KPI

### 9.1 Impacts

| Domaine                | Impact                                                            |
|------------------------|-------------------------------------------------------------------|
| Performance            | Métriques versionnées, Grafana, +8,3 % sur deux runs comparables  |
| Robustesse             | Framing, limite 1 Mo, gestion d’erreurs, agrégation des résultats |
| Sécurité               | Rust, YARA-X, réduction du sudo, validation du protocole          |
| Maintenabilité         | Crates partagées, contrat minimal, GUI déclarative, guides        |
| Temps de développement | Coût initial du socle, puis réutilisation pour chaque plugin      |
| Ouverture              | RSS/Discord, Reddit, dépôt public et workflow de contribution     |

### 9.2 Matrice de conformité

| Exigence                    | Preuve                                    | État    |
|-----------------------------|-------------------------------------------|---------|
| Veille active               | RSS, Discord, GitHub, sources officielles | Couvert |
| Analyse comparative         | ABI, IPC, YARA-X, GUI                     | Couvert |
| Benchmark                   | JSON, SQLite, Grafana                     | Couvert |
| 1–2 POC                     | Plugin Manager et IPC v1                  | Couvert |
| Intégration                 | abi_stable, CBOR, YARA-X, Tauri           | Couvert |
| Mise à jour du projet       | Code, docs, packaging, releases           | Couvert |
| Participation communautaire | Post r/rust et dépôt public               | Couvert |
| Synthèse d’impact           | Performance, sécurité, maintenance        | Couvert |

**Le dossier répond aux objectifs obligatoires. Les actions restantes renforcent surtout la reproductibilité et la traçabilité, pas la couverture du KPI.**

## 10. Limites et prochaines actions

| Priorité | Action                                             | Bénéfice                                     |
|----------|----------------------------------------------------|----------------------------------------------|
| P0       | Valideur TOML <-> init() <-> handle_message()      | Détecter les incohérences avant installation |
| P0       | Pagination et limites métier                       | Éviter les réponses non bornées              |
| P0       | Export SQL/CSV automatique des benchmarks          | Rendre les résultats auditable               |
| P1       | Versionnement explicite de l'ABI et du protocole   | Éviter de casser les plugins                 |
| P1       | Documenter licences et sources des règles YARA-X   | Sécuriser la supply chain                    |
| P1       | Tests reproductibles : machine, cache, répétitions | Renforcer la validité scientifique           |
| P2       | POC WebAssembly / WASI                             | Évaluer une sandbox future                   |

### Conclusion

Griffon démontre une démarche complète de veille technologique : un problème est identifié, plusieurs solutions sont comparées, un POC est réalisé, la solution retenue est intégrée, puis ses effets sont mesurés et documentés. Les choix `abi_stable`, `Frame` + `CBOR`, `YARA-X`, `Tauri` et `Grafana` répondent à des contraintes concrètes du projet et non à un simple effet de tendance.

Le passage du prototype monolithique à une plateforme modulaire constitue la preuve principale. Les mesures du `Cleaner`, l'incident des 10 000 chemins, les PR, les guides, la veille RSS et le post Reddit montrent que la démarche est appliquée au code, à la qualité et au partage.

**À présenter au jury : architecture modulaire, comparaison des choix, deux POC, incident des 10 000 chemins, benchmark Cleaner et preuves de partage.**

### Références principales

Rust Reference — `ABI` ; `abi_stable` ; RFC 8949 `CBOR` ; `YARA-X` documentation ; `Tauri` architecture/security ; `Grafana` dashboards ; documentation et historique GitHub Griffon.

Preuves GitHub principales : PR #90, #91, #118, #122, #149, #175, #180 ; issues `Cleaner` #80–89, #134 ; Scanner #130.